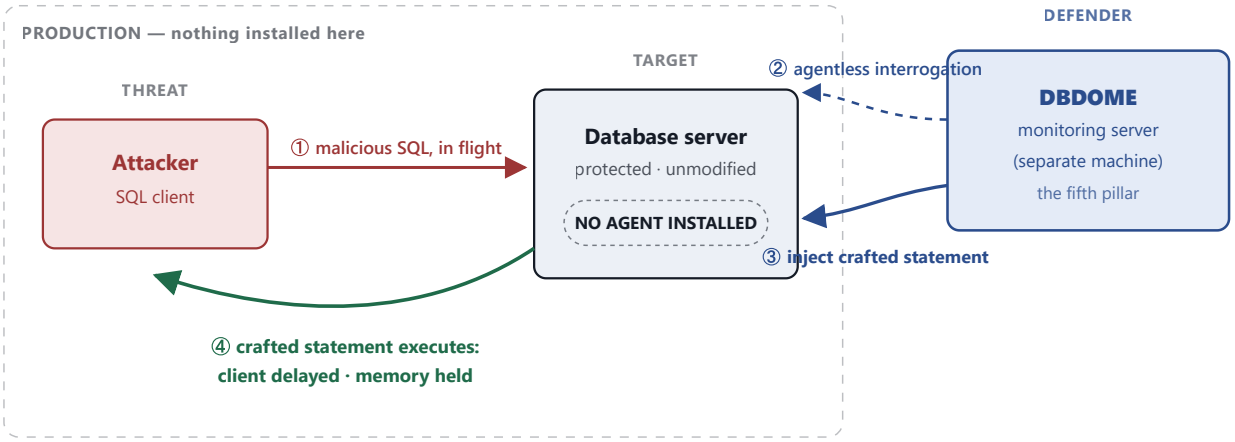
DBDOME · TECHNICAL EXPLAINER

Deterrence by Design

How the method works, element by element — and the measurement that proves it does something the prior art cannot.

01 The mechanism, end to end

Four parties, one boundary. The database being protected is a normal, unmodified server. The defender — DBDOME — sits *outside* that server and reaches it the same way any client does. The whole invention lives in what happens along the numbered path below.



The boundary is the point. DBDOME never enters the production server — steps ② and ③ cross into it only through the database's own query path, the same channel a normal client uses. Because the defender has no foothold inside the server, neutralization **must** be expressed as a database statement (step ③) rather than as code running on the host. That constraint is what forces the crafted-statement mechanism into existence.

02 The four elements, in technical detail

No element alone is the invention; the combination is. Each is defined below against what it concretely requires, and against the specific prior-art technique it departs from.

Element 1 Agentless — where the defender sits

WHAT IT REQUIRES

Interrogation and neutralization are performed from a **separate monitoring server**, over an ordinary database client connection. No driver, plug-in, instrumentation, or sidecar is installed on the protected host — nothing to deploy, patch, or maintain on the production database.

TECHNICAL CONSEQUENCE

Because the defender holds no process inside the server, it cannot intercept the statement in the host's memory. It must act **through the query interface itself** — which is exactly why neutralization takes the form of a substituted SQL statement rather than a hooked function call. The constraint produces the mechanism.

DEPARTS FROM

Gupta (Virsec) requires "instrumented code on the web server … or the application server executing the web application." Its detection *depends* on that agent; remove it and the method has no input.

Element 2 In-flight — when the alteration happens

WHAT IT REQUIRES

The client's statement is altered **while it is executing**, inside the live session — the "while occurring" window. Not pre-execution filtering, not post-hoc audit of a log.

HOW IT IS DONE

On SQL Server, the active request is identified from the live-session views (`dm_exec_sessions`, `dm_exec_requests`, `dm_exec_sql_text`), and the neutralization is applied to that session while its request is still open.

DEPARTS FROM

Alberstein (Verizon) sanitizes *before* execution. **FairWarning** applies rules to *stored* audit logs, after the fact. Both act outside the execution window; this element acts inside it.

Element 3 **Dynamic** how the replacement is chosen

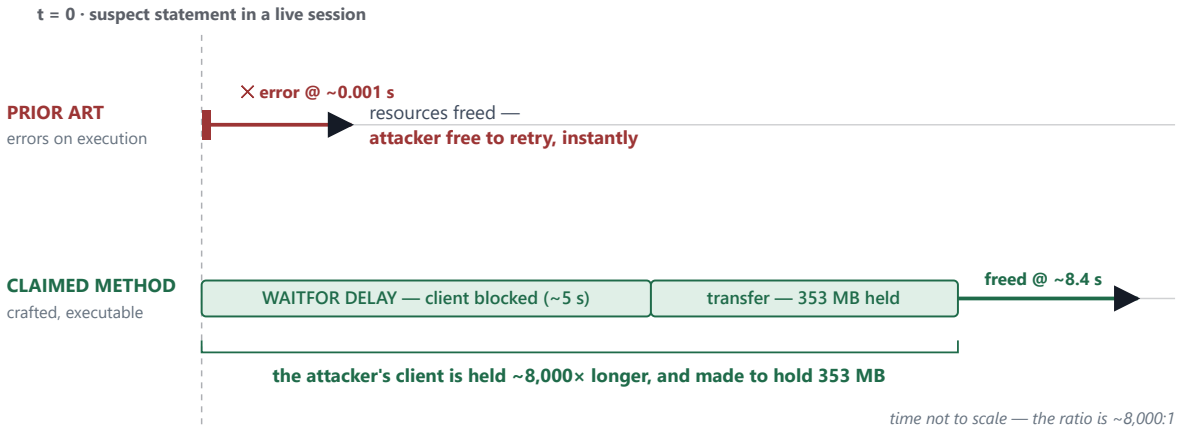
WHAT IT REQUIRES	The crafted statement is **composed in dependence on the identified activity** — its shape, its delay, and the memory it forces can vary per attack, and differ between successive occurrences.
TECHNICAL CONSEQUENCE	The response is tunable to the threat: a longer stall for a persistent scanner, a heavier memory load for a bulk-extraction attempt. It is not one canned transform applied to everything.
DEPARTS FROM	**Alberstein** applies a **fixed** transform — delimiting unknown elements as inert identifiers, uniformly. The Office Action's own citation for the "dynamic" limitation is silent on any per-attack variation.

Element 4 **Sustained cost** what the crafted statement does

WHAT IT REQUIRES	The crafted statement remains **executable**, and its execution **delays the client and holds the client's memory**. The attacker's own process is what is consumed.
THE TWO LEVERS	**Delay** — a wait function (e.g. `WAITFOR DELAY`) holds the client's connection blocked. **Memory** — a large materialized result set forces the client's driver to buffer megabytes it cannot release until the request completes.
DEPARTS FROM	**Every cited approach** ends the request as fast as possible — error out, kill the session, restore a backup. An erroring statement *releases* resources; it cannot delay or hold. This element inverts the goal: keep it running, at the attacker's expense.

03 What happens to the attacker's client

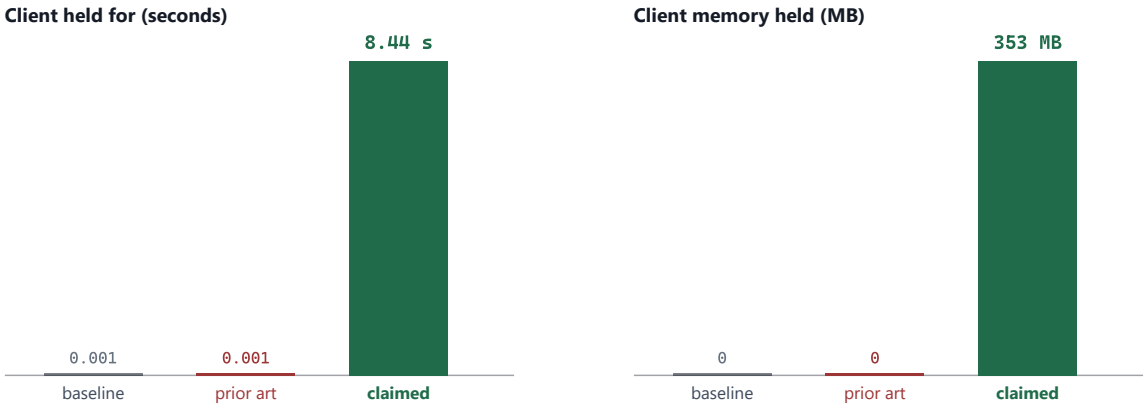
The clearest way to see the invention is to put the two neutralizations on the same clock. The prior-art path and the claimed path start identically — a suspect statement enters a live session — and then diverge completely.



Same start, opposite outcome. The prior-art neutralization succeeds by *failing fast*: the statement errors, resources release, and the attacker learns immediately and cheaply. The claimed method succeeds by *lasting*: the substituted statement runs, holding the connection through a deliberate delay and forcing the client to buffer a large result — a cost the attacker pays in time and memory. The blocks are the two levers of Element 4.

04 The measurement

Argument becomes evidence when it is measured. The claimed method and the closest prior-art technique were each run five times against a live Microsoft SQL Server 2022 instance. Bars show the median.



The two left bars in each chart are drawn as hairlines because the measured values are, to scale, invisible.
That is the point: a difference not of degree, but of kind.

Median of five runs, live SQL Server 2022, 15 Aug 2026. The prior-art neutralization ended in about one millisecond, holding no measurable client memory — the statement errored (`42S02`) and released everything. The claimed method held the client for **8.44 s** and **353 MB**, consistently. The 8.44 s comprises a deliberate ~5 s delay plus the transfer of a large result set the client is

forced to buffer; both are components of "delays operation of the client." Full per-run figures and the runnable program are in the accompanying exhibit.

ARM	STATEMENT EXECUTED BY THE CLIENT	MEDIAN HELD	MEDIAN MEMORY	OUTCOME
A	Baseline — the attacker's statement, unmodified	0.001 s	0 MB	completes
B	Prior art — statement altered so it errors on execution	0.001 s	0 MB	SQL error 42S02
C	**Claimed method** — statement replaced with a crafted, executable statement	8.44 s	353 MB	completes

> **The prior art releases everything in a millisecond. The claimed method holds the attacker's client for more than eight seconds and 353 megabytes — the exact effect an error-producing sanitizer cannot produce.**

---

defender path (DBDOME) attacker / prior-art fast-exit claimed effect — cost imposed on the client

Reference characterizations drawn from the published texts of US 2022/0272121 (Verizon), US 2016/0337400 (Virsec Systems) and US 2021/0216666 (Pure Storage). Measurements produced against a Microsoft SQL Server 2022 test instance; reproducible via the accompanying experiment program. Explanatory summary — the claims define the invention.